1D MSRE Modelling

Yuyao

September 11, 2024

System components

- Core - Neutronics and Thermal-hydraulics
- Heat exchanger 1 and 2
- Transfer delay
- Feedback - Reactivity

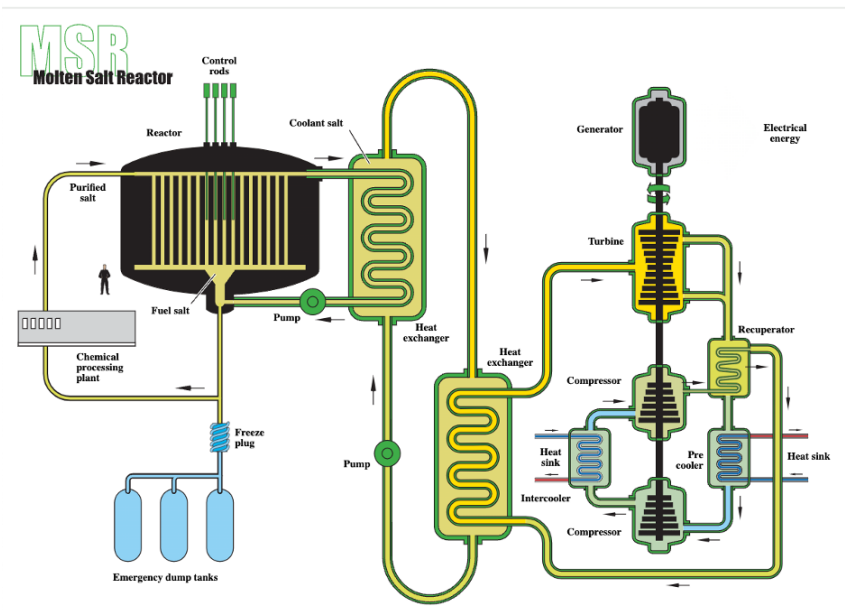

Figure 1: MSRE

Equations

0.1 Neutronics

$$\frac{1}{V}\frac{\partial \phi(x,t)}{\partial t} = D\frac{\partial^2 \phi(x,t)}{\partial x^2} + \left(-\Sigma_a + \frac{(1-\beta)\nu\Sigma_f}{K_eff}\right)\phi(x,t) + \sum_{i=1}^{6}\lambda_i c_i(x,t) \tag{1}$$

$$\frac{\partial c_i(x,t)}{\partial t} = \beta\nu\Sigma_f\phi(x,t) - \lambda_i c_i(x,t) \tag{2}$$

$$K_{eff} = \frac{1}{1-\rho} \tag{3}$$

0.2 Thermal-hydraulics

$$q'(x,t) = \frac{\phi(x,t)\Sigma_f V}{3.12\times 10^{10}\ \frac{\text{fissions}}{\text{watt-sec}}} \tag{4}$$

$$\frac{\partial T_s(x,t)}{\partial t} + v_c\frac{\partial T_s(x,t)}{\partial x} = \frac{H}{M_s c_{p,s}}\left(\gamma q'(x,t) + \frac{U}{H}\left(T_g(x,t) - T_s(x,t)\right)\right) \tag{5}$$

$$\frac{\partial T_g(x,t)}{\partial t} = \frac{H}{M_g c_{p,g}}\left((1-\gamma)q'(x,t) - \frac{U}{H}\left(T_g(x,t) - T_s(x,t)\right)\right) \tag{6}$$

0.3 Reactivity feedback

$$\rho(t) = \rho_0 + \rho_{fb}(t) + \rho_{insertion}(t) \tag{7}$$

$$\rho_0 = \beta - \sum_{i=1}^{6}\frac{\beta_i}{\left(1 + \frac{1}{\lambda_i\tau_C}\left[1 - e^{-\lambda_i\tau_L}\right]\right)} \tag{8}$$

$$\rho_{fb}(t) = \alpha_f\left[T_f(x,0) - T_f(x,t)\right] + \alpha_g\left[T_g(x,0) - T_g(x,t)\right] \tag{9}$$

0.4 Heat exchanger 1 and 2

$$\frac{\partial T_s^{h.e.}(z,t)}{\partial t} + v_s^{h.e.}\frac{\partial T_s^{h.e.}(z,t)}{\partial z} = \frac{U^{h.e.}}{M_s^{h.e.}\cdot c_{p,s}}\left(T_s^{h.e.}(z,t) - T_{ss}(z,t)\right) \tag{10}$$

$$\frac{\partial T_{ss}(z,t)}{\partial t} + v_{ss}^{h.e.}\frac{\partial T_{ss}(z,t)}{\partial z} = \frac{U^{h.e.}}{M_{ss}^{h.e.}\cdot c_{p,ss}}\left(T_s^{h.e.}(z,t) - T_{ss}(z,t)\right) \tag{11}$$

Equations for heat exchanger 2 is similar to that for heat exchanger 1.

Methods

In a time loop:

- *dvalue_dt*: discrete the space into grid points (By multiplying a matrix)
- *updated_value*: solve the time domain differential equations (*solve_ivp*: explicit method like $RK23$ is fast but not stable, implicit method like Radau can be stable but slow)
- passing the updated value to the next function

Results

When the system is stable, reactivity should be 0 ($K_{eff} = 1$), and the neutron flux and DN precursors should be stable.
To get results faster, they are solved by $RK23$, but the results are generally similar to the implicit method, except for some oscillations.

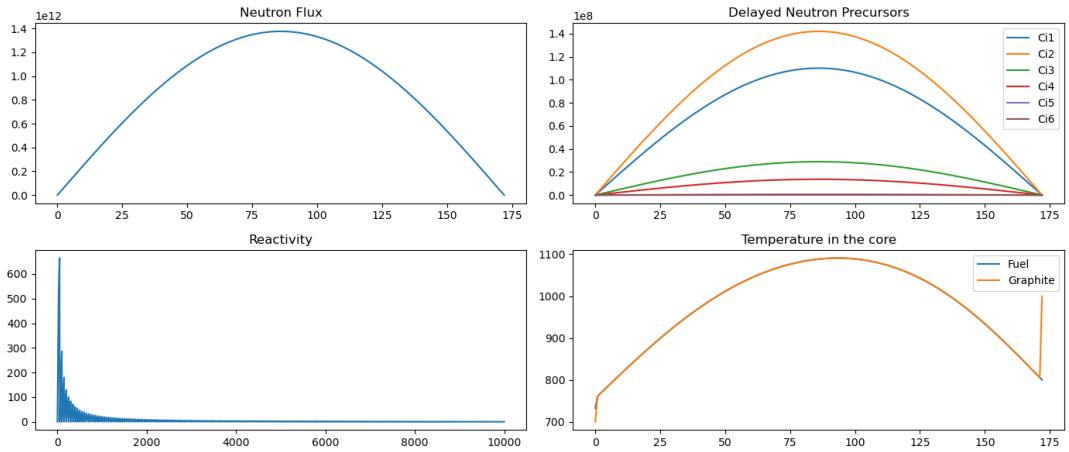

Figure 2: No insertion

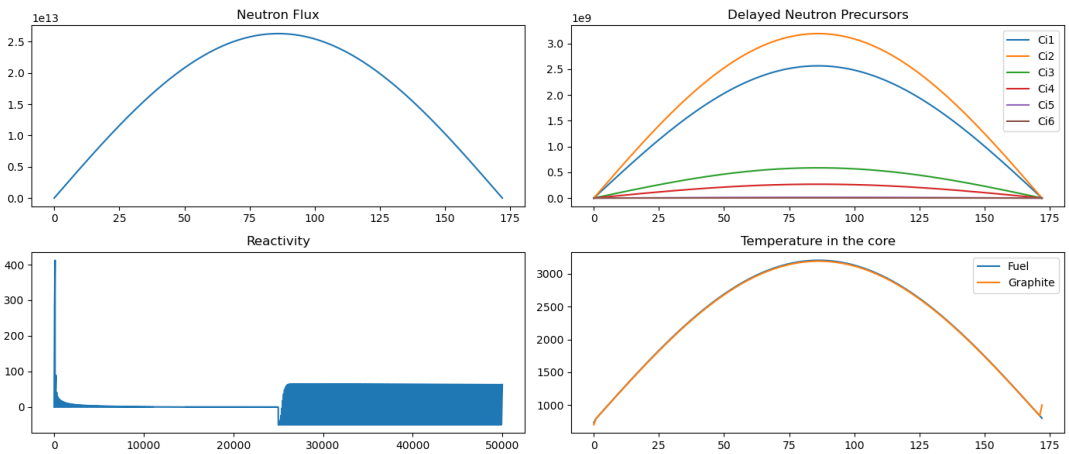

Figure 3: 50 pcm reactivity insertion at time=25000

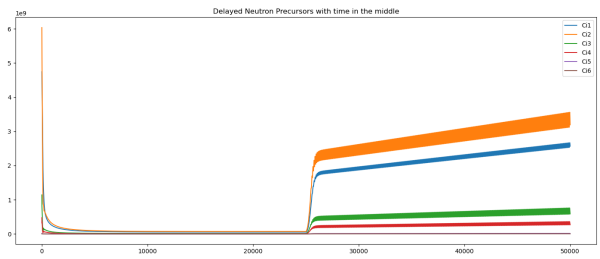

Figure 4: precursor with time middle (with insertion at time=25000)

Problems to fix

- The model is too sensitive to some parameters, which makes the model too sensitive to reactivity
- A very small reactivity insertion can cause the model to diverge, which makes rod control impossible
- The temperature of graphite should be a little higher than the fuel salt, but the model always shows the identical temperature of graphite and fuel salt

Possible solutions:

- Continue trying modifying parameters
- The reactivity model may be not accurate (very few articles I can find about 1D reactivity feedback model, and the equation I'm using now comes from a 0D model, and I just expanded it to vector directly)
- The automatic converter from python to c++ may cause some problems, as Yulin suggested, I should all use pointers to pass values to let FPGA smoothly run the model in the further work, but automatic converters usually use values to pass values
- I plan to convert the model to c++ after the model is well tested in python including the rode insertion control part, as c++ is not easy to debug